A Research-Oriented Framework for Evaluating Synthetic NLP Datasets Using Fidelity, Diversity, and Downstream Utility

Raunak Singh Baweja and Om Nikam
NLP Evaluation Research Project

***Abstract*—This paper presents a research-oriented framework for evaluating synthetic natural language processing (NLP) datasets against their original counterparts. Rather than treating synthetic-data quality as a single similarity measure, the proposed system evaluates three complementary dimensions: fidelity, diversity, and downstream utility. Fidelity is assessed using dataset statistics, text-length characteristics, vocabulary and n-gram overlap, TF-IDF similarity, semantic similarity, and class-distribution alignment. Diversity is assessed using unique-text ratio, duplicate rate, vocabulary richness, and repetition indicators. Utility is measured through a controlled classification experiment in which a model trained on synthetic data is evaluated on the same held-out real test set used for the real-data baseline. The framework was implemented as a full-stack application with a React/TypeScript frontend, FastAPI backend, PostgreSQL experiment storage, and an NLP evaluation engine based on Python and scikit-learn, with sentence-embedding support. A Hindi three-class sentiment dataset was used for experimental evaluation. The evaluated original dataset contained 2,479 records and the synthetic dataset contained 2,480 records. The experiment produced a project-defined composite score of 53/100, with fidelity of 41%, diversity of 22%, and a normalized utility score of 100%. The raw downstream macro-F1 retention was approximately 120%. The results illustrate why downstream utility alone is insufficient: despite strong class-distribution alignment and high utility, the synthetic dataset exhibited very low vocabulary overlap and a substantially higher duplicate rate. The framework therefore provides a transparent and reproducible basis for comparing synthetic NLP datasets.**

***Index Terms*—Synthetic data, NLP, dataset evaluation, text classification, fidelity, diversity, utility, sentiment analysis, TF-IDF, semantic similarity.**

I. Introduction

Synthetic data is increasingly used to augment machine-learning datasets, reduce dependence on costly annotation, and support privacy-aware development. In NLP, however, a synthetic corpus can appear plausible while still failing to reproduce the statistical and linguistic structure required by downstream models. Conversely, a dataset may obtain strong task performance while being overly repetitive or too close to the source distribution. This creates a need for evaluation procedures that separate different notions of quality rather than collapsing them into a single measure.

The practical appeal of synthetic text has grown rapidly with the arrival of large pretrained language models. Transformer architectures [1] and pretrained encoders such as BERT [2] made high-quality text representations widely available, while few-shot generative models [3] made it possible to produce entire labeled corpora from a handful of prompts. Consequently, practitioners now routinely generate training data for classification, instruction following, and dialogue [4–7]. Earlier data-augmentation approaches such as synonym replacement, random insertion, and back-translation [8, 9], as well as conditional generation with pretrained transformers [10, 11], have shown that artificially produced examples can improve classifier accuracy, particularly in low-resource settings.

This convenience introduces a methodological problem. When a synthetic dataset is used in place of, or alongside, real data, researchers must be able to state *in what sense* it is good. A corpus that faithfully copies the original may offer little novelty and may even leak sensitive training records [12]. A corpus that is highly varied may drift away from the real task distribution. A corpus that supports high classifier accuracy may nevertheless be dominated by a small number of templates, which limits its value for any task other than the one that was measured. Recent work emphasizes that synthetic-data quality is multidimensional, with quality, diversity, and downstream effects interacting in non-trivial ways [13], and that generated corpora frequently exhibit reduced diversity and systematic bias [14].

The problem is sharper for languages other than English. Most evaluation tooling, embedding models, and benchmarks are developed primarily for high-resource languages, whereas the majority of the world's languages remain comparatively underserved [15]. Indian languages such as Hindi are written in the Devanagari script, are morphologically rich, and frequently appear in code-mixed form, so tokenization and lexical statistics behave differently from English. Resources such as IndicNLP-Suite have begun to close this gap [16], and surveys of synthetic data for multilingual NLP highlight both the usefulness of generated examples and the difficulty of identifying unsuitable or low-quality instances [17]. These observations motivate a practical evaluation framework in which a real dataset and a synthetic dataset pass through the same analysis pipeline, with language-agnostic preprocessing and clearly documented metrics.

The objective of this work is therefore to design and implement a reusable NLP evaluation platform that answers a simple

research question: how closely does a synthetic dataset reproduce the characteristics and usefulness of an original dataset? The proposed system evaluates three dimensions—fidelity, diversity, and utility—and reports both individual metrics and a transparent project-defined composite score. More specifically, the work addresses three sub-questions: (RQ1) does the synthetic corpus preserve the lexical, distributional, and semantic properties of the original corpus; (RQ2) does it contain sufficient internal variety, or is it dominated by duplicates and repeated phrases; and (RQ3) can a classifier trained only on synthetic data perform well on real held-out data?

The main contributions of this paper are as follows:

- A three-dimensional evaluation formulation that separates fidelity, diversity, and downstream utility, with every component metric defined by an explicit formula.
- A controlled Synthetic→Real versus Real→Real utility protocol in which both models are evaluated on the identical held-out original test split.
- A full-stack, reproducible implementation with dataset validation, aligned metric computation, experiment history, and CSV/JSON/PDF reporting.
- An experimental evaluation on a Hindi three-class sentiment corpus that demonstrates how high utility can coexist with poor lexical fidelity and diversity.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Section IV presents the methodology and implementation, including all metric formulas. Section V describes the dataset and experimental setup, and Section VI reports and discusses the results. Section VII lists limitations, and Section VIII concludes the paper.

II. Related Work

A. Synthetic Data Generation for NLP

Synthetic text has been produced by rule-based augmentation, paraphrasing, and, more recently, by prompting pretrained language models. Wei and Zou introduced simple edit-based augmentation operations that improve text classifiers on small datasets [8], and Feng *et al.* surveyed the broader landscape of NLP augmentation strategies [9]. Conditional generation with pretrained transformers [10] and language-model-based augmentation [11] extended this idea to label-conditioned generation. Large-scale generators have since been used to create whole datasets for zero-shot learning [4–6] and to bootstrap instruction data [7]. Jordon *et al.* provide a general overview of why and how synthetic data is produced across domains [18]. These works focus primarily on generation; evaluation is typically limited to the downstream accuracy of the target task.

B. Evaluation of Synthetic and Generated Data

Evaluation of synthetic data has traditionally considered statistical similarity and downstream predictive performance. Recent synthetic-text research increasingly argues for multi-dimensional evaluation, including utility, distributional differences, diversity, fairness, and privacy-related risks [19]. The quality–diversity trade-off is also important: maximizing similarity to a reference corpus does not necessarily maximize useful novelty [13]. For generative models more generally, Alaa *et al.* proposed sample-level fidelity, diversity, and generalization metrics [20], and the Fréchet Inception Distance compares feature-space distributions for images [21]. In natural-language generation, reference-based metrics such as BLEU [22] and BERTScore [23] measure overlap with references, while MAUVE compares the distributions of machine and human text using divergence frontiers [24]. These metrics were designed for generation quality rather than for auditing labeled training corpora, which motivates the dataset-level approach adopted here.

C. Lexical and Distributional Similarity

For textual representations, TF-IDF remains a simple and interpretable representation for comparing lexical characteristics and supporting baseline classifiers [25, 26]. Cosine similarity is commonly used with TF-IDF vectors to compare documents or corpus representations [27]. The Jaccard coefficient [28] provides a bounded measure of set overlap that is well suited to comparing vocabularies and n-gram inventories. Jensen–Shannon divergence provides a symmetric, bounded way to compare probability distributions and is therefore suitable for class-distribution or other distributional comparisons [29].

D. Semantic Representations

For semantic analysis, sentence embeddings provide a practical way to compare meaning beyond exact lexical overlap; Sentence-BERT introduced efficient sentence-level embeddings that can be compared with cosine similarity [30]. Multilingual extensions trained by knowledge distillation map many languages into a shared embedding space [31], which makes it possible to apply the same semantic measure to Hindi and other non-English corpora.

E. Diversity, Repetition, and Memorization

Diversity in generated text is commonly measured with distinct-n ratios [32] and Self-BLEU [33]. Duplicate and near-duplicate content is also a known quality issue in training corpora: Lee *et al.* showed that deduplicating training data improves language models and reduces memorization [34], while Carlini *et al.* demonstrated that language models can reproduce verbatim training sequences [12]. Generated datasets are particularly prone to repetition and attribute bias [14]. These findings justify treating diversity as an independent evaluation dimension rather than as a by-product of fidelity.

F. Downstream Classification and Metrics

Linear models over sparse text features remain strong and fast baselines for text classification [35]. Logistic regression [36], support vector machines [37], and multinomial naive Bayes [38] are standard choices, and macro-averaged precision, recall, and F1 are widely recommended for multi-class evaluation because they weight every class equally [39].

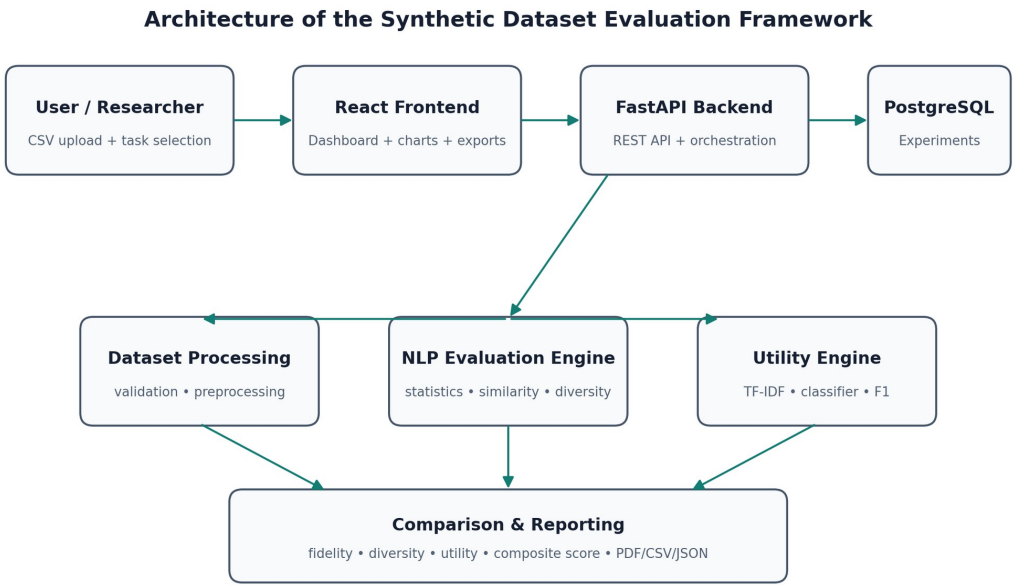


Fig. 1. System architecture of the proposed synthetic NLP dataset evaluation framework. The upper row shows the request path from the researcher to storage; the middle row shows the three analysis engines; the bottom block aggregates their outputs into scores and reports.

G. Research Gap and Positioning

Existing studies either focus on generation methods and report only task accuracy, or propose individual metrics without an integrated workflow. The contribution of this project is not a new generative model. Instead, it is an integrated evaluation workflow that combines interpretable linguistic statistics, distributional similarity, diversity diagnostics, and a controlled downstream utility experiment in one reproducible application.

III. System Architecture

The framework follows a layered architecture. A researcher uploads an original CSV and a synthetic CSV through the web interface. The frontend communicates with a FastAPI backend, which validates the files, identifies text and label columns, executes the NLP evaluation pipeline, and persists experiment metadata and results. PostgreSQL stores dataset metadata, metric values, comparison records, and configuration information.

The evaluation engine is divided into dataset processing, linguistic evaluation, similarity/diversity analysis, and utility evaluation. The comparison layer then converts the two evaluation outputs into aligned metric pairs and computes differences or normalized similarity scores. Finally, the reporting layer presents an interactive dashboard and supports CSV, JSON, and PDF exports.

A. Detailed Explanation of the Architecture Diagram

Fig. 1 is organized into three horizontal tiers, and the arrows indicate the direction of data and control flow.

Tier 1 – Interaction and orchestration. The left-most block, *User/Researcher*, represents the person running an evaluation; their inputs are two CSV files and a task selection (text classification in the current version). The arrow into the *React Frontend* indicates that all interaction happens through a browser-based dashboard, which also renders charts and triggers exports. The next arrow represents REST calls from the frontend to the *FastAPI Backend*, which exposes endpoints for uploading datasets, creating comparisons, retrieving comparison details, and downloading reports. The backend acts as the orchestrator: it validates requests, invokes the analysis engines in the correct order, and writes results through an object-relational mapping layer. The arrow to *PostgreSQL* shows that every experiment is persisted, including the dataset records, per-dataset evaluation runs, per-metric values, and the full configuration used, so that an experiment can be reopened from the history page and reproduced later.

Tier 2 – Analysis engines. The diagonal arrow from the backend to the *NLP Evaluation Engine* and the horizontal branches to its left and right show that one comparison request fans out into three cooperating components. *Dataset Processing* performs validation and preprocessing: it checks for missing values and duplicates, normalizes whitespace, and tokenizes text. The central *NLP Evaluation Engine* computes dataset statistics, similarity measures, and diversity indicators. The *Utility Engine* builds TF-IDF features, trains classifiers, and computes F1-based metrics. Separating these engines allows each to be tested and extended independently; for example, a new task-specific utility model can be added without changing the lexical metrics.

Tier 3 – Aggregation and reporting. The three converging arrows into *Comparison & Reporting* indicate that the outputs of all engines are merged into aligned original-versus-synthetic metric pairs, grouped into fidelity, diversity, and utility, and combined into the composite score. The same block generates

the natural-language interpretation and the PDF, CSV, and JSON exports.

The architecture intentionally separates dataset-independent metrics from task-dependent metrics. General textual metrics can be reused across datasets, while utility metrics are selected according to the NLP task. The initial implementation targets text classification and is demonstrated with sentiment analysis.

IV. Methodology and Implementation

This section combines the evaluation methodology with its implementation. Each metric is described together with its formula and the component in which it is computed. Notation: D_o and D_s denote the original and synthetic datasets, $n_o = |D_o|$ and $n_s = |D_s|$ their sizes, V_o and V_s their vocabularies, and $\epsilon = 10^{-9}$ a small constant that prevents division by zero. All similarity values lie in $[0, 1]$ and are multiplied by 100 when displayed as percentages.

A. Dataset Ingestion and Preprocessing

Dataset ingestion begins with CSV validation, missing-value inspection, duplicate detection, and user confirmation of the text and label columns. The same preprocessing configuration is applied to both datasets. Raw text is preserved for semantic analysis, while normalized/tokenized representations are used for lexical statistics.

Normalization collapses consecutive whitespace and trims each record. Tokenization is Unicode-aware: after lowercasing, a character is kept as part of a token when its Unicode general category is a letter (L), mark (M), or number (N). Retaining the *mark* category is essential for Devanagari, where vowel signs (matras) and the virama are combining marks; an ASCII-only tokenizer would split Hindi words into fragments. Stopwords are not removed by default so that informative task vocabulary is not discarded.

B. Dataset Statistics

For a dataset with documents $d_1, \ldots, d_n$, let ℓ_i be the number of tokens in d_i and $N = \sum_i \ell_i$ the total token count. The mean text length and the type-token ratio (TTR) are

$$\mu = \frac{1}{n}\sum_{i=1}^{n} \ell_i, \qquad \mathrm{TTR} = \frac{|V|}{N}. \tag{1}$$

TTR is defined as the number of unique tokens divided by the total number of tokens. Median, standard deviation, minimum, and maximum length, word-frequency tables, top unigrams and bigrams, and the class proportions $p(c) = n_c/n$ are also recorded.

For scalar statistics such as record count, mean length, vocabulary size, and TTR, the comparison layer converts the original value a and synthetic value b into a bounded similarity:

$$S_{\mathrm{rel}}(a, b) = \max\!\left(0,\ 1 - \frac{|a-b|}{\max(|a|, \epsilon)}\right). \tag{2}$$

This form is used because a larger or smaller value is not inherently better; only the deviation from the original matters.

C. Fidelity Metrics

Fidelity is evaluated using record count, mean text length, vocabulary size, type-token ratio (TTR), unigram/bigram overlap, TF-IDF similarity, semantic similarity, and class-distribution similarity.

1) Vocabulary and n-gram overlap. Vocabulary overlap is based on set intersection and union using the Jaccard coefficient [28]:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{3}$$

It is applied to the vocabularies ($S_{\mathrm{voc}} = J(V_o, V_s)$), to unigram sets ($S_{1g}$), and to the sets of distinct bigrams ($S_{2g} = J(G_o^{(2)}, G_s^{(2)})$).

2) TF-IDF similarity. A single vectorizer (unigrams and bigrams, up to 10,000 features) is fitted on the union of both corpora so that both share one feature space. The weight of term t in document d is

$$w_{t,d} = \mathrm{tf}_{t,d} \cdot \left(\ln \frac{1+n}{1+\mathrm{df}_t} + 1\right), \tag{4}$$

followed by L_2 normalization of each document vector [25, 27]. The dataset centroids $\bar{\mathbf{x}}_o$ and $\bar{\mathbf{x}}_s$ are the means of the document vectors, and the similarity is their cosine:

$$S_{\mathrm{tfidf}} = \cos(\bar{\mathbf{x}}_o, \bar{\mathbf{x}}_s) = \frac{\bar{\mathbf{x}}_o \cdot \bar{\mathbf{x}}_s}{\|\bar{\mathbf{x}}_o\|\,\|\bar{\mathbf{x}}_s\|}. \tag{5}$$

3) Semantic similarity. A random sample (fixed seed) of each dataset is encoded with a sentence-embedding model [30, 31] into L_2-normalized vectors $\mathbf{e}_i$. The semantic similarity is the cosine between the embedding centroids,

$$S_{\mathrm{sem}} = \cos\!\Big(\tfrac{1}{m_o}\textstyle\sum_i \mathbf{e}_i^o,\ \tfrac{1}{m_s}\sum_j \mathbf{e}_j^s\Big). \tag{6}$$

If the embedding model is unavailable, the system falls back to the TF-IDF centroid similarity and records that fallback in the experiment configuration.

4) Class-distribution similarity. Let P and Q be the original and synthetic label distributions and $M = \frac{1}{2}(P+Q)$. The Jensen–Shannon divergence [29] with base-2 logarithms is

$$\mathrm{JSD}(P\|Q) = \tfrac{1}{2}\mathrm{KL}(P\|M) + \tfrac{1}{2}\mathrm{KL}(Q\|M), \tag{7}$$

where $\mathrm{KL}(P\|M) = \sum_c P(c)\log_2 \frac{P(c)}{M(c)}$. The divergence is bounded in $[0, 1]$ and is converted to a similarity through the Jensen–Shannon distance:

$$S_{\mathrm{cls}} = 1 - \sqrt{\mathrm{JSD}(P\|Q)}. \tag{8}$$

The same construction is applied to character 3-gram distributions as a supplementary surface-form divergence indicator.

5) Fidelity score. The fidelity score is the mean of seven bounded components:

$$F = \frac{100}{7}(S_{\mathrm{voc}} + S_{1g} + S_{2g} + S_{\mathrm{tfidf}} + S_{\mathrm{sem}} + S_{\mathrm{cls}} + S_{\mathrm{len}}), \tag{9}$$

where $S_{\mathrm{len}} = S_{\mathrm{rel}}(\mu_o, \mu_s)$ from (2).

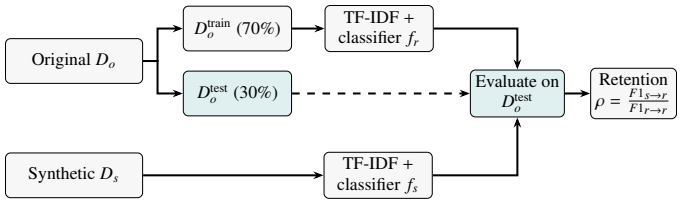


Fig. 2. Controlled utility protocol. The original data are split once (stratified, fixed seed); the real-trained model f_r and the synthetic-trained model f_s use the same vectorizer and classifier family and are both scored on the identical real test split, which is never used for training.

D. Diversity Metrics

Diversity is evaluated independently because high fidelity can coexist with poor novelty. The framework calculates the unique-text ratio, duplicate rate, vocabulary richness, and repeated phrase indicators:

$$U = \frac{|\{\text{distinct texts}\}|}{n}, \qquad R_{\text{dup}} = \frac{n_{\text{dup}}}{n}, \tag{10}$$

where n_{dup} counts records whose normalized text already occurred earlier in the dataset. Phrase repetition is measured with the repeated trigram rate

$$R_{3g} = \frac{\sum_{g:\, c(g)>1} c(g)}{\sum_g c(g)}, \tag{11}$$

where $c(g)$ is the frequency of trigram g. The diversity score averages five components:

$$\begin{aligned} D = \frac{100}{5}(U_s + (1 - R_{\text{dup},s}) + \text{TTR}_s \\ + S_{\text{rel}}(U_o, U_s) + S_{\text{rel}}(R_{3g,o}, R_{3g,s})). \end{aligned} \tag{12}$$

A synthetic corpus containing many repeated templates can therefore receive a low diversity score even if its class distribution is correct.

E. Downstream Utility Experiment

Downstream utility is evaluated using a controlled experiment, illustrated in Fig. 2. First, a classifier is trained on the real training data and tested on a held-out real test set. The same model configuration is then trained on synthetic training data and evaluated on the identical real test set. This produces a direct estimate of how useful synthetic data is for approximating the original task. The baseline implementation uses TF-IDF features with conventional linear classification models, with accuracy, precision, recall, macro-F1, and weighted-F1 recorded.

In Fig. 2, the upper path is the Real→Real baseline and the lower path is the Synthetic→Real experiment. The dashed arrow emphasizes that the 30% real test split flows only into evaluation. Three classifier families are run in both paths: logistic regression with balanced class weights [36], a linear support vector machine [37], and multinomial naive Bayes [38]. For each class c with precision P_c and recall R_c,

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{K}\sum_{c=1}^{K} F1_c, \tag{13}$$

Algorithm 1 Synthetic dataset comparison pipeline

Input: Original CSV D_o, synthetic CSV D_s, weights w, seed σ
Output: Metric table, scores $F, D, U_{\text{util}}, S_{\text{overall}}$, report

```
Validate files; confirm text and label columns
for X ∈ {D_o, D_s} do
    Normalize and tokenize; compute μ, |V|, TTR, U, R_dup, R_3g, p(c)
end for
Compute S_voc, S_1g, S_2g, S_tfidf, S_sem, S_cls, S_len
Split D_o into 70/30 train/test (stratified, seed σ)
for each model family k do
    Train f_r on D_o^train, f_s on D_s; score both on D_o^test; compute ρ_k
end for
Compute F (9), D (12), U_util, and S_overall (15)
Persist configuration and metrics; render dashboard; export report
```

where K is the number of classes [39]. For each model family k, the macro-F1 retention is

$$\rho_k = \min\left(\frac{F1^{(k)}_{\text{macro}}(s\to r)}{\max\left(F1^{(k)}_{\text{macro}}(r\to r), \epsilon\right)},\ 1.2\right), \tag{14}$$

and the utility score uses the best model, $U_{\text{util}} = 100 \cdot \min(1, \max_k \rho_k)$. Utility retention may exceed 100% when synthetic-trained performance is higher than the real-data baseline; the dashboard normalizes the dimension score to a maximum of 100 while retaining the raw retention value for interpretation.

F. Composite Score

The project-defined composite score combines the three dimensions using weights of 40% fidelity, 30% diversity, and 30% utility:

$$S_{\text{overall}} = w_F F + w_D D + w_U U_{\text{util}}, \tag{15}$$

with $(w_F, w_D, w_U) = (0.4, 0.3, 0.3)$. User-supplied weights are renormalized to sum to one. The score is explicitly a project metric rather than a universal scientific standard. Scores of 80 or more are labeled strong, 60–79 moderate, and below 60 limited alignment, which the dashboard displays as "Needs Review."

G. Implementation

The complete procedure is summarized in Algorithm 1. The user interface was implemented as a research dashboard with five workflow stages: dataset type, original dataset, synthetic dataset, evaluation, and results. The application provides upload validation, progress indicators, metric cards, comparison charts, experiment history, methodology documentation, and export controls. The frontend is written in React and TypeScript with Vite, Tailwind CSS for styling, TanStack Query for server-state management, and Recharts for visualizations.

The backend is organized into dataset, preprocessing, NLP metric, comparison, utility-model, and reporting services. It is built with FastAPI and Pydantic for typed request/response schemas, SQLAlchemy for persistence, pandas and NumPy for data handling, SciPy for the Jensen–Shannon computation, scikit-learn [27] for vectorization and classification, and ReportLab for PDF generation. PostgreSQL is used for experiment storage, with SQLite supported for local development. Expensive

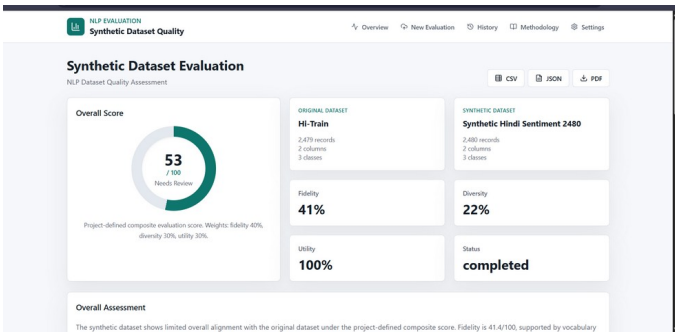


Fig. 3. Evaluation dashboard showing the overall score and the three evaluation dimensions.

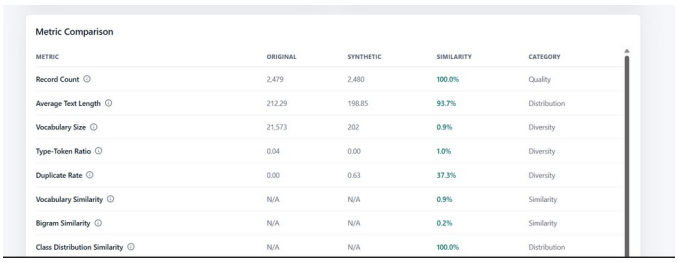


Fig. 4. Metric-level comparison between the original and synthetic Hindi sentiment datasets.

operations such as embedding generation can be batched and executed through background processing. Experiment configurations record model names, random seeds, sampling parameters, preprocessing settings, metric definitions, and scoring weights to improve reproducibility.

The application deliberately avoids building a synthetic-data generator. The synthetic dataset is treated as an external input so that the framework can evaluate datasets produced by different generation methods without coupling evaluation to a particular generator. Unit tests cover the metric functions and the API.

V. Dataset Description and Experimental Setup

A Hindi sentiment-analysis dataset was selected to evaluate the framework. The experiment used three labels—negative, neutral, and positive—and compared the original corpus (labeled *Hi-Train* in the dashboard) with a synthetic corpus of approximately equal size (*Synthetic Hindi Sentiment 2480*). The interface reported 2,479 records for the original dataset and 2,480 for the synthetic dataset. Both datasets contained two columns (text and label) and three classes. As shown later in Fig. 5, the label proportions are roughly 30% negative, 28% neutral, and 42% positive in both corpora.

All metrics were computed with the configuration described in Section IV: Unicode-aware tokenization without stopword removal, a shared TF-IDF vectorizer with unigrams and bigrams (maximum 10,000 features), a stratified 70/30 split of the original data with a fixed random seed, and three linear classifier families. Composite weights were $(0.4, 0.3, 0.3)$. The results should be interpreted as a demonstration of the evaluation framework rather than a claim about synthetic-data generation in general. The experiment used one dataset pair and therefore cannot establish broad statistical conclusions across languages, domains, or generation methods.

VI. Results and Discussion

A. Description of the Dashboard Screens

Evaluation dashboard (Fig. 3). The header contains the navigation bar (Overview, New Evaluation, History, Methodology, and Settings) and three export buttons for CSV, JSON, and PDF. The left card shows the overall score as a donut gauge: the filled arc corresponds to 53 out of 100, and the label "Needs Review" reflects the limited-alignment band. The caption below the gauge restates the weights so that the number is never shown without its definition. The two cards at the top right identify the compared datasets (*Hi-Train*, 2,479 records, and *Synthetic Hindi Sentiment 2480*, 2,480 records, each with two columns and three classes). The four smaller cards report the dimension scores—fidelity 41%, diversity 22%, utility 100%—and the run status, *completed*.

Metric comparison table (Fig. 4). Each row lists one metric with its original value, synthetic value, similarity, and category (quality, distribution, diversity, or similarity). Paired statistics such as record count and text length show both raw values, whereas overlap metrics such as vocabulary and bigram similarity are inherently pairwise and show "N/A" in the value columns. The information icon next to each metric opens its definition and formula, which keeps the interpretation transparent.

Assessment and charts (Fig. 5). The *Overall Assessment* panel contains the automatically generated interpretation. Below it, the *Class Distribution* chart plots the original (teal) and synthetic (orange) proportions for each label side by side; the bars are almost identical in height. The *Dimension Scores* chart shows fidelity, diversity, and utility on a 0–100 scale, with the tooltip revealing the exact diversity value of 22.42. The visual contrast between the short diversity bar and the full utility bar communicates the main finding at a glance.

B. Quantitative Results

The results show a clear separation between the evaluation dimensions. The average text length was 212.29 for the original data and 198.85 for the synthetic data, producing a reported similarity of 93.7%. Class-distribution similarity was reported as 100.0%, indicating that the synthetic data reproduced the label proportions closely.

However, lexical diversity was substantially weaker. The original vocabulary contained 21,573 unique tokens compared with 202 in the synthetic dataset. The resulting reported vocabulary similarity was only 0.9%, while bigram similarity was 0.2%. TTR was 0.04 for the original dataset and approximately 0.00 for the synthetic dataset. Using the mean lengths, the original corpus contains roughly 526,000 tokens and the synthetic corpus roughly 493,000, so a comparable amount of text is generated from a vocabulary that is about 100 times smaller. The duplicate rate increased from approximately 0.00 in the original dataset to 0.63 in the synthetic dataset, corresponding to a low duplicate-

TABLE I
SUMMARY OF EXPERIMENTAL RESULTS

Metric	Original vs. Synthetic / Result
Record count	2,479 vs. 2,480 (100.0%)
Mean text length	212.29 vs. 198.85 (93.7%)
Vocabulary size	21,573 vs. 202 (0.9%)
Type-token ratio	0.04 vs. approx. 0.00 (1.0%)
Duplicate rate	approx. 0.00 vs. 0.63 (37.3%)
Bigram similarity	0.2%
Class-distribution similarity	100.0%
Fidelity score	41%
Diversity score	22%
Utility score	100% normalized; raw macro-F1 retention approx. 120%
Overall composite	53 / 100

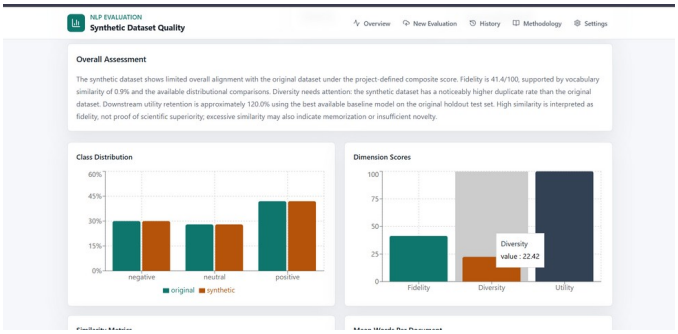


Fig. 5. Overall assessment and dimension scores generated by the evaluation framework.

rate similarity of 37.3%; in other words, only about 37% of the synthetic records are distinct texts. These results indicate substantial repetition and limited lexical novelty in the synthetic corpus.

The dashboard reported a project-defined fidelity score of 41% (41.4), diversity of 22% (22.42), and a normalized utility score of 100%, producing an overall composite score of 53/100. Substituting into (15) gives $0.4(41.4) + 0.3(22.42) + 0.3(100) = 53.3$, which is displayed as 53. The raw downstream macro-F1 retention was approximately 120%. Thus, the synthetic dataset performed strongly in the selected downstream experiment despite exhibiting weak lexical fidelity and diversity. This is an important result: task utility alone would have overstated the quality of the synthetic corpus.

C. Discussion

The central result is that synthetic-data quality is not one-dimensional. The experiment achieved excellent class-distribution alignment and high downstream utility, but poor vocabulary overlap and diversity. The 93.7% text-length similarity suggests that coarse document-length characteristics were reasonably preserved, while the 0.9% vocabulary similarity reveals that the synthetic corpus did not reproduce the lexical variety of the original corpus. Table I summarizes these values.

The difference between utility and diversity is particularly informative. A classifier can learn the target label boundary from repeated or highly templated examples even when those examples cover only a small portion of the original vocabulary. Consequently, the reported 120% macro-F1 retention should not be interpreted as evidence that the synthetic dataset is superior to the original. Instead, it shows that the synthetic examples were sufficient for the selected classification task under the particular experimental setup.

The breakdown of the scores explains why the composite is only moderate. In the fidelity score (9), the class-distribution and length components are close to one, but the three overlap components are close to zero, which pulls the average down to 41.4. In the diversity score (12), the unique-text ratio and $(1 - R_{\text{dup}})$ are both about 0.37 and the synthetic TTR is near zero, which yields 22.42. Only the utility dimension saturates.

The results also demonstrate the value of an integrated dashboard. The same experiment simultaneously exposes a high utility score and a low diversity score, encouraging a researcher to inspect the data rather than relying on a single headline number. For research reporting, this provides a more defensible description of synthetic-data behavior.

VII. LIMITATIONS

Several limitations remain. First, the demonstration is based on a single Hindi sentiment dataset and one synthetic dataset, so the findings are not generalizable to all NLP tasks. Second, lexical similarity metrics are sensitive to tokenization and language-specific preprocessing. Third, semantic similarity based on pre-trained embeddings depends on the selected embedding model and may not capture every linguistic or cultural nuance of Hindi text.

Fourth, the composite score uses project-defined weights. Although the weighting makes the dashboard easy to interpret, different applications may reasonably assign different importance to fidelity, diversity, and utility. Finally, downstream utility does not constitute a privacy guarantee, and near-duplicate detection is only an indicator of potential overlap rather than a definitive privacy-leakage test.

VIII. CONCLUSION AND FUTURE WORK

This work presents a practical, research-oriented framework for evaluating synthetic NLP datasets through three complementary dimensions: fidelity, diversity, and downstream utility. The implemented system provides a consistent workflow for uploading real and synthetic datasets, calculating aligned metrics, running controlled classification experiments, and generating interpretable comparison reports.

The Hindi sentiment evaluation demonstrates the value of the approach. Although the synthetic corpus closely matched class distributions and produced strong downstream utility, its very low vocabulary similarity and high duplicate rate revealed important quality deficiencies that would have been missed by task accuracy alone. The final project-defined score of 53/100 therefore provides a more nuanced assessment than a single predictive metric.

Future work can extend the framework to additional NLP tasks such as topic classification, spam detection, named-entity recognition, and multilingual evaluation. Further research can add

stronger privacy-risk analysis, fairness evaluation, human quality assessment, confidence intervals across repeated trials, and comparisons among multiple synthetic-data generation methods. A larger benchmark spanning languages and domains would also enable statistically stronger conclusions.

References

[1] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[3] T. B. Brown *et al.*, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[4] T. Schick and H. Schütze, "Generating datasets with pretrained language models," in *Proc. EMNLP*, 2021, pp. 6943–6951.

[5] Y. Meng, J. Huang, Y. Zhang, and J. Han, "Generating training data with language models: Towards zero-shot language understanding," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022.

[6] J. Ye *et al.*, "ZeroGen: Efficient zero-shot learning via dataset generation," in *Proc. EMNLP*, 2022.

[7] Y. Wang *et al.*, "Self-Instruct: Aligning language models with self-generated instructions," in *Proc. ACL*, 2023, pp. 13484–13508.

[8] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 6382–6388.

[9] S. Y. Feng *et al.*, "A survey of data augmentation approaches for NLP," in *Findings of ACL-IJCNLP*, 2021, pp. 968–988.

[10] V. Kumar, A. Choudhary, and E. Cho, "Data augmentation using pre-trained transformer models," in *Proc. 2nd Workshop on Life-long Learning for Spoken Language Systems*, 2020, pp. 18–26.

[11] A. Anaby-Tavor *et al.*, "Do not have enough data? Deep learning to the rescue!" in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 5, 2020, pp. 7383–7390.

[12] N. Carlini *et al.*, "Extracting training data from large language models," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 2633–2650.

[13] A. Havrilla *et al.*, "Surveying the effects of quality, diversity, and complexity in synthetic data from large language models," arXiv preprint arXiv:2412.02980, 2024.

[14] Y. Yu *et al.*, "Large language model as attributed training data generator: A tale of diversity and bias," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, vol. 36, 2023.

[15] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, "The state and fate of linguistic diversity and inclusion in the NLP world," in *Proc. ACL*, 2020, pp. 6282–6293.

[16] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of EMNLP*, 2020, pp. 4948–4961.

[17] M. Doshi and P. Bhattacharyya, "Synthetic data for multilingual NLP: A survey," CFILT, Indian Institute of Technology Bombay, 2024.

[18] J. Jordon *et al.*, "Synthetic data—what, why and how?" arXiv preprint arXiv:2205.03257, 2022.

[19] K. Ramesh *et al.*, "SynthTextEval: Synthetic text data generation and evaluation for high-stakes domains," arXiv preprint arXiv:2507.07229, 2025.

[20] A. Alaa, B. van Breugel, E. S. Saveliev, and M. van der Schaar, "How faithful is your synthetic data? Sample-level metrics for evaluating and auditing generative models," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 162, 2022, pp. 290–306.

[21] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[22] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting ACL*, 2002, pp. 311–318.

[23] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[24] K. Pillutla *et al.*, "MAUVE: Measuring the gap between neural text and human text using divergence frontiers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021.

[25] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[26] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[28] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.

[29] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inf. Theory*, vol. 37, no. 1, pp. 145–151, 1991.

[30] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[31] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proc. EMNLP*, 2020, pp. 4512–4525.

[32] J. Li, M. Galley, C. Brockett, J. Gao, and B. Dolan, "A diversity-promoting objective function for neural conversation models," in *Proc. NAACL-HLT*, 2016, pp. 110–119.

[33] Y. Zhu *et al.*, "Texygen: A benchmarking platform for text generation models," in *Proc. 41st Int. ACM SIGIR Conf.*, 2018, pp. 1097–1100.

[34] K. Lee *et al.*, "Deduplicating training data makes language models better," in *Proc. ACL*, 2022, pp. 8424–8445.

[35] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proc. EACL*, 2017, pp. 427–431.

[36] D. R. Cox, "The regression analysis of binary sequences," *J. Roy. Statist. Soc. B*, vol. 20, no. 2, pp. 215–242, 1958.

[37] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[38] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, 1998, pp. 41–48.

[39] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, 2009.